



git

--distributed-is-the-new-centralized

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git : Notes Index

Page	Title	Topics
1	Git : Introduction & Installation	What is Git, VCS, Git vs GitHub, Install & Config
2	Git : Basic Git Concepts	Repository, Working Directory, Staging Area, Commit, Snapshot, Workflow
3	Git : Configure Git Settings	git config - set name & email, list config, edit config
4	Git : Starting a Project	git init - initialize repo, git clone - clone repo
5	Git : Basic Snapshotting	git status - check state, git add - stage changes
6	Git : Basic Snapshotting – II	git diff - view changes, git commit - save changes
7	Git : Basic Snapshotting – III	git restore - undo changes, git reset - roll back commits
8	Git : Basic Snapshotting – IV	git rm - delete file, git mv - rename or move file
9	Git : Branching and Merging	git branch - create, list, delete branch, What is a branch
10	Git : Branching and Merging – II	git switch - switch branch, git switch -c - create & switch
11	Git : Branching and Merging – III	git merge - merge branch, Resolve merge conflicts
12	Git : Branching and Merging – IV	git stash - save work temporarily
13	Git : Branching and Merging – V	git log - view commit history
14	Git : Branching and Merging – VI	git tag - mark a version / release
15	Git : Remote and Team Work	git remote - connect to GitHub, git push - upload changes
16	Git : Remote and Team Work – II	git pull - download & merge, git fetch - download only
17	Git : Inspection Advanced	git diff - compare branches & commits
18	Git : Inspection Advanced – II	git show - inspect a commit in detail
19	Git : Undoing Advanced	git revert - undo a pushed commit
20	Git : Undoing Advanced – II	git rebase - clean up history
21	Git : Undoing Advanced – III	git cherry-pick - apply one commit to current branch
22	Git : Ignore Files	.gitignore - ignore files from tracking
23	Git : Daily Workflow	Local workflow - solo, Remote workflow - solo with GitHub
24	Git : Daily Workflow – II	Team workflow - with GitHub

What is : Git

Git is a **distributed version control system (VCS)** that helps you track changes in your files and collaborate with others efficiently. It allows multiple people to work on a project without overwriting each other's work.

Think of Git as a way to:

- Keep a history of all changes in your project.
- Revert back to previous versions if something goes wrong.
- Collaborate with others by merging changes from different team members.

Importance of : Version Control Systems

Before version control systems, people would manually save copies of files with names like: project_v1, project_v2, or project_final_final_revised.

This approach is messy and error-prone. Git solves these problems:

- 1 Tracks changes automatically:** No need to rename files.
- 2 Collaborates easily:** Multiple people can work on the same file simultaneously.
- 3 Saves time:** Quickly identify who made changes and why.

Difference Between : Git and GitHub

- Git is the tool for version control on your computer.
- GitHub is a cloud-based platform where you can store your Git repositories and collaborate with others online.

Installing Git : In Windows

1. Download Git from the official site: git-scm.com.
2. Run the installer.
3. Select the default options (or customize if needed).
4. Verify installation: Open Command Prompt and type

```
1 git --version
```

Setting up : Git

After installing Git, configure your username and email (these will appear in your commit history):

1. Open your terminal.
2. Run the following commands

```
2 git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

```
3 git config --list (Verify your settings)
```

Basic : Git Concepts

Before diving into Git commands, it's important to understand how Git works conceptually. This will help you grasp what happens when you use Git commands.

Key Concepts : in Git

1. Repository

A Git repository is where Git tracks changes in your project. It's essentially a folder where Git saves:

- The current version of your files.
- The history of all previous versions
- Metadata about the changes.
- Local Repository: Stored on your computer.
- Remote Repository: Stored on a server (e.g., GitHub, GitLab).

2. Working Directory, Staging Area, and Commit History

Git operates with three main areas:

1 Working Directory:

- This is your actual project folder.
- Any file you create, modify, or delete is part of the working directory.

2 Staging Area:

- Before committing changes, you need to add them to the staging area.
- Think of it as a "to-do list" of changes that Git will save.

3 Commit History:

- After staging changes, you commit them. A commit is a snapshot of your project at a specific point in time.
- Git keeps a history of all commits.

3. Snapshot-Based Version Control

- Git takes "snapshots" of your files.
- When you commit, Git saves a snapshot of every tracked file at the time of commit. If a file hasn't changed, Git doesn't store it again — it just links to the previous identical version. This makes Git efficient without being diff-based.

4. SHA-1 Hash

- Every commit in Git is assigned a unique identifier called a hash.
- Example: f4a7c8e93a9473d8ad8a6a43236c153b84f21e09.

Hashes make it easy to reference specific changes.

How Git Works Overview of Workflow

1 Modify Files in your working directory

2 Stage Changes:

- Add files to the staging area with `git add`.

3 Commit Changes:

- Save the staged changes to the commit history with `git commit`.

4 (Optional) Push Changes:

- Send commits to a remote repository like GitHub.

Set Your Identity : Git config

Used to view, add, change, or remove Git configuration settings at the system, user, or repository level.

- | | | |
|---|---|---|
| 1 | <code>git config --global user.name "Your Name"</code> | # Set your name |
| 2 | <code>git config --global user.email "email@example.com"</code> | # Set your email |
| 3 | <code>git config --list</code> | # Verify everything is set |
| 4 | <code>git config --global --edit</code> | # Open the global config file in editor |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
# set your name and email

$ git config --global user.name "Anjesh Kumar"

$ git config --global user.email "anjeshravi@yahoo.com"

$ git config --list

diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.email=anjeshravi@yahoo.com
user.name=Anjesh Kumar
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true

$ git config --global --edit

[user]
  email = anjeshravi@yahoo.com
  name = Anjesh Kumar

~/ .gitconfig [unix] (00:54 29/05/2026)
AllType :qa and press <Enter> to exit Vim
```

Initialize a Repository : **git init**

Creates a new Git repository in the current directory, allowing Git to start tracking files and changes in the project.

- | | | |
|---|-----------------|--|
| 1 | git init | # Initialize a new Git repository |
| 2 | dir -a | # Verify it worked — .git folder should appear |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
$ git init
Initialized empty Git repository in D:/LearnCode/Git/.git/

$ dir -a
.  ..  .git  Code.bash  hello.txt

# a hidden .git folder is now created # Git will now track all changes in this folder
```

Download a Repository : **git clone**

Creates a local copy of an existing Git repository, including its files, commit history, and branch information.

- | | | |
|---|---|-----------------------------------|
| 1 | git clone https://github.com/Kumaravi-admin/git-repo.git | # Download a project from GitHub |
| 2 | git clone https://github.com/Kumaravi-admin/git-repo.git my-folder | # Clone into a custom folder name |
| 3 | cd git-repo | # Go into the downloaded project |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
# clone repository
$ git clone https://github.com/Kumaravi-admin/git-repo.git
Cloning into 'git-repo'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.

# clone repository in specific folder
$ git clone https://github.com/Kumaravi-admin/git-repo.git my-folder
Cloning into 'my-folder'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.

# change directory
$ cd git-repo
```

Check Repository State : **git status**

Shows the current state of the repository, including modified files, staged changes, and untracked files.

1 **git status** # See what is modified, staged, or untracked

2 **git status -s** # Short version — cleaner output

```
Git Bash
$ cd /d/LearnCode/Git # Move to the project working directory
-----

$ git status
On branch master
No commits yet
Untracked files: (use "git add <file>..." to include in what will be committed)
    Code.bash
    hello.txt

$ git status -s
?? Code.bash
?? hello.txt

# M = modified
# A = added to staging
# ?? = untracked (Git doesn't know about this file yet)
```

Stage Changes : **git add**

Adds new or modified files to the staging area, preparing them to be included in the next commit.

1 **git add hello.txt** # Stage a specific file

2 **git add hello.txt bye.txt** # Stage multiple files

3 **git add .** # Stage everything at once (most common)

4 **git add src/** # Stage an entire folder

```
Git Bash
$ cd /d/LearnCode/Git # Move to the project working directory
-----

# creat hello.txt and bye.txt files
$ touch hello.txt bye.txt

# add hello.txt
$ git add hello.txt

# add hello.txt and bye.txt
$ git add hello.txt bye.txt

# create src folder
$ mkdir src

# add src folder
$ git add src/

# add add all folder and files
$ git add .
```

View Changes : **git diff**

Shows the differences between files, commits, or changes in the working directory and staging area.

- | | | |
|---|-----------------------------|---|
| 1 | git diff | # Changes in working directory NOT yet staged |
| 2 | git diff --staged | # Changes that ARE staged (ready to commit) |
| 3 | git diff --name-only | # Show only file names that changed |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# show changes in the working directory that are NOT yet staged
$ git diff
index 3c7fba8..0000000
+++ /dev/null
@@ -1,10 +0,0 @@
-$ git status
-On branch master
-No commits yet
-Untracked files:
-(use "git add <file>..." to include in what will be committed)
- hello.txt
-$ git status -s
-?? hello.txt
\ No newline at end of file

# show differences between the staging area (index) and the last commit
$ git diff --staged
index 6204f16..3c7fba8 100644
@@ -1,17 +1,10 @@
-$ git add hello.txt
-$ git add .
-$ git add src/

# show the names of files that have changed
$ git diff --name-only
```

Save Changes : **git commit**

Takes a snapshot of everything in the staging area and saves it to history.

- | | | |
|---|--|---|
| 1 | git commit -m "Add hello.txt" | # commit with a message |
| 2 | git commit -am "Update hello.txt" | # stage + commit in one step (only tracked files) |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

$ git commit -m "Add hello.txt"
[master (root-commit) 188ea87] Add hello.txt
2 files changed, 17 insertions(+)
create mode 100644 Code.bash
create mode 100644 hello.txt

$ git commit -am "update hello.txt"
[master 57363a6] update hello.txt
1 file changed, 1 insertion(+)
```


Undo Local Changes : git restore

Used to discard changes in files and restore them to a previous state, either from the staging area or the last commit.

- 1 **git restore --staged hello.txt** # Unstage a file — undo git add
- 2 **git restore hello.txt** # File goes back to how it was in last commit
- 3 **git restore .** # Undo all changes in entire working directory

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
# unstage a file - undo git add
$ git restore --staged hello.txt

# throw away all changes in a file permanent
# file goes back to how it was in last commit
$ git restore hello.txt

# undo all changes in entire working directory
$ git restore .
```

Roll Back Commits : git reset

Moves HEAD to a previous commit and can undo commits, with options to keep or remove changes from the staging area and working directory depending on the mode used.

- 1 **git reset --soft HEAD~1** # Remove last commit but KEEP your changes staged (safe)
- 2 **git reset --mixed HEAD~1** # Remove last commit and put changes back in working directory
- 3 **git reset --hard HEAD~1** # Remove last commit and DELETE all changes permanent
- 4 **git reset --soft HEAD~3** # go back 3 commits (keep changes) require minimum 3 commits

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
$ git init
echo "A" > file.txt
git add . git commit -m "commit 1"

# add a second commit
$ echo "B" >> file.txt
git add . git commit -m "commit 2"

# add a third commit
$ echo "C" >> file.txt
git add . git commit -m "commit 3"

# remove last commit but KEEP your changes staged (safe)
$ git reset --soft HEAD~1

# # remove last commit and put changes back in working directory
$ git reset --mixed HEAD~1
Unstaged changes after reset: M      file.txt

# remove last commit and DELETE all changes permanent
$ git reset --hard HEAD~1
HEAD is now at fe0f063 commit 1
```

Delete a File : **git rm**

Removes a file from Git tracking and the working directory (or only from Git tracking if used with `--cached`). The file is deleted only if it matches the latest committed version, and it helps keep the repository and staging area in sync.

① **git rm hello.txt** # Delete file from project and Git tracking both

② **git rm --cached hello.txt** # Save the deletion with a commit

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Delete file from project and Git tracking both
$ git rm hello.txt
rm 'hello.txt'

# Make file and add to Git for testing git rm --cached command
$ echo "Hello Git Learner" > hello.txt
$ git add hello.txt

# Remove from Git tracking only – keep file on disk
# useful for files you forgot to add in .gitignore
$ git rm --cached hello.txt
rm 'hello.txt'

# Save the deletion with a commit
$ git commit -m "Remove hello.txt"
[master cbbd2d4] Remove hello.txt
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 hello.txt
```

Rename or Move a File : **git mv**

Moves or renames files, directories, or symlinks in the repository. It updates the Git index automatically, but the change still needs to be committed.

① **git mv old-name.txt new-name.txt** # Rename a file

② **git mv hello.txt src/hello.txt** # Move a file into a folder, folder must already exist

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Create a file named hello.txt for test command
$ echo "hello git learner" > hello.txt

# Rename the file hello.txt to helloGitLearner.txt
$ git mv hello.txt helloGitLearner.txt

# Create a folder if not exist
$ mkdir src

# Move a file into a folder, folder must already exist if create folder first
$ git mv helloGitLearner.txt src/hello.txt

# Commit changes
$ git commit -m "Move hello.txt into src folder"
[master ad226e4] Move hello.txt into src folder
1 file changed, 1 insertion(+)
create mode 100644 helloGitLearner.txt
```

What is a : Branch in Git

A branch in Git is a separate line of development. It lets you work on features or changes without affecting the main code.

Think of your project like a tree:

main/master ----> main trunk (stable code)

branch ----> side path where you try new things

Why branches are used

Work on new features safely, Fix bugs without breaking main code, Test ideas separately, Multiple developers can work in parallel

Manage Branches : git branch

Lists, creates, or deletes branches in a Git repository. By default, it shows local branches and highlights the current one. It can also display remote branches or all branches using specific options.

- | | | |
|---|--|---|
| 1 | <code>git branch feature-login</code> | # Create a new branch |
| 2 | <code>git branch</code> | # list all branches (* marks the current one) |
| 3 | <code>git branch -a</code> | # List both local and remote branches |
| 4 | <code>git branch -d feature-login</code> | # Delete a branch after it has been merged |
| 5 | <code>git branch -D feature-login</code> | # Force delete — even if not merged |

Git Bash

```
$ cd /d/LearnCode/Git      # Move to the project working directory
```

```
# Create a new branch
```

```
$ git branch feature-login
```

```
# list all branches (* marks the current one)
```

```
$ git branch
feature-login
```

```
* master
```

```
# list both local and remote branches
```

```
$ git branch -a
feature-login
```

```
* master
```

```
# delete a branch after it has been merged
```

```
$ git branch -d feature-login
Deleted branch feature-login (was ad226e4).
```

```
# force delete — even if not merged
```

```
$ git branch -D feature-login
Deleted branch feature-login (was ad226e4).
```

Change Branches : git switch

Switches to another branch and updates your working files to match it. You can also create and switch to a new branch using -c. New commits will be added to the currently active branch.

- | | | |
|---|--|--|
| 1 | <code>git switch feature-login</code> | # Switch to an existing branch |
| 2 | <code>git switch -c feature-payment</code> | # Create a new branch and switch to it immediately (most used) |
| 3 | <code>git switch -</code> | # Go back to the previous branch |
| 4 | <code>git checkout feature-login</code> | # Old way — still works but switch is cleaner |

Git Bash

```
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Create a new branch name feature-login (if it doesn't exist)
$ git branch feature-login

# Switch to existing branch (in this case feature-login)
$ git switch feature-login
A      hello.txt
D      helloGitLearner.txt
A      src/hello.txt
Switched to branch 'feature-login'

# Create a new branch name feature-payment and switch to it
$ git switch -c feature-payment
Switched to a new branch 'feature-payment'

# Switch back to feature-login
$ git switch -
A      hello.txt
D      helloGitLearner.txt
A      src/hello.txt
Switched to branch 'feature-login'

# Switch to the feature-login branch (old way; git switch is preferred)
$ git checkout feature-login
A      hello.txt
D      helloGitLearner.txt
A      src/hello.txt
Already on 'feature-login'
```

Merge Branches : git merge

Combines changes from another branch into the current branch.

It is commonly used to bring completed work from a feature branch into the main branch.

- | | | |
|---|--|---|
| 1 | <code>git switch main</code> | # Step 1 — always switch to main first |
| 2 | <code>git merge feature-login</code> | # Step 2 — merge your feature branch |
| 3 | <code>git merge --no-ff feature-login</code> | # Always create a merge commit (preserves branch history) |
| 4 | <code>git branch -d feature-login</code> | # Step 3 — delete the branch after merging (cleanup) |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Create a main branch (if not exist)
$ git branch main

# Switch to the main branch (always switch to the main branch first)
$ git switch main
A       hello.txt
D       helloGitLearner.txt
A       src/hello.txt
Switched to branch 'main'

# Merge the feature-login branch into the main branch
$ git merge feature-login
Already up to date.

# Always create a merge commit (preserves branch history)
$ git merge --no-ff feature-login
Already up to date.

# Delete a branch after successful merge
$ git branch -d feature-login
Deleted branch feature-login (was ad226e4).
```

Merge Conflict : Resolve Conflicts

A merge conflict occurs when Git cannot automatically combine changes from different branches. You must manually review the conflicting code, choose the correct version (or combine both), and then complete the merge.

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

git merge feature-login     # attempt the merge
git status                  # Git will tell you which files have conflicts

# open the conflicted file — you will see this inside:
# <<<<<< HEAD               <- your code (main branch)
# your version of code
# =====
# their version of code
# >>>>>> feature-login      <- incoming branch

# step 1 — remove the conflict markers
# step 2 — keep the correct code
# step 3 — stage the resolved file
git add hello.txt

# step 4 — commit the resolution
git commit -m "Resolve conflict in hello.txt"

# want to cancel the merge and go back?
git merge -abort
```

Save Work Temporarily : git stash

Temporarily saves your uncommitted changes and restores the working directory to a clean state. You can later view, apply, or recover the saved changes when you're ready to continue working.

- | | | |
|---|---|--|
| 1 | <code>git stash save "login form work in progress"</code> | # Save unfinished work (cleans your working directory) |
| 2 | <code>git stash list</code> | # See all saved stashes |
| 3 | <code>git stash pop</code> | # Bring back your latest stash and remove it from list |
| 4 | <code>git stash apply stash@{1}</code> | # Apply a specific stash without removing it |
| 5 | <code>git stash drop stash@{0}</code> | # Delete a stash you no longer need |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Create a file hello.txt to test stash commands
$ echo Hello World > hello.txt
$ git add hello.txt
$ git commit -m "Initial commit"

# Make some uncommitted changes
$ echo "Hello Git Learner" >> hello.txt

# Save unfinished work (cleans your working directory)
$ git stash save "login form work in progress"
Saved working directory and index state On main: login form work in progress

# See all saved stashes
$ git stash list
stash@{0}: On main: login form work in progress

# Restore the latest stash and remove it from the stash list
$ git stash pop
On branch main
Changes not staged for commit:
  modified: hello.txt
Dropped refs/stash@{0}

# Apply the stash
-----
# Make some changes again
$ echo "More work in progress" >> hello.txt

# Save changes to a new stash
$ git stash save "login form work in progress"

# Restore the stash but keep it in the stash list
$ git stash apply stash@{0}

# Delete the stash manually
$ git stash drop stash@{0}
Dropped stash@{0}
```

View Commit History : git log

Displays the history of commits in a repository, including commit IDs, authors, dates, and messages. Various options can be used to customize how the commit history and changes are displayed.

1	git log	# Basic History
2	git log --oneline	# Compact — one line per commit
3	git log --oneline --graph --all	# Visual graph showing all branches
4	git log -5	# Show last 5 commits only
5	git log --oneline hello.txt	# See history of one specific file
6	git log --author="Your Name"	# See all commits by a specific person

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Show the complete commit history with details
$ git log
commit 57363a64a7e2bfdae3c4617596223a9dd060e4a (HEAD -> master)
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date:   Fri May 29 02:15:41 2026 +0530

    update hello.txt

commit 188ea87716f325bd870bd787967a8aa8d6524775
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date:   Fri May 29 02:14:34 2026 +0530

    Add hello.txt

# Show a compact commit history (one commit per line)
$ git log --oneline
57363a6 (HEAD -> master) update hello.txt
188ea87 Add hello.txt

# Display a visual graph of all branches and commits
$ git log --oneline --graph --all
* 57363a6 (HEAD -> master) update hello.txt
* 188ea87 Add hello.txt

# Show only the last 5 commits
$ git log -5
commit 57363a64a7e2bfdae3c4617596223a9dd060e4a (HEAD -> master)
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date:   Fri May 29 02:15:41 2026 +0530

    update hello.txt

# Show commit history for a specific file
$ git log --oneline hello.txt
57363a6 (HEAD -> master) update hello.txt
188ea87 Add hello.txt

# Show commits created by a specific author
$ git log --author="Anjesh Kumar"

commit e354a8d3f1a04a3dd4edf60b144d5a146b8ff178 (HEAD -> main)
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date:   Fri May 29 23:57:31 2026 +0530

    Initial commit
```

Mark a Version : git tag

Creates, lists, verifies, or deletes tags in a Git repository. Tags are commonly used to mark important commits, such as releases and version numbers (e.g., v1.0.0), making them easy to reference later.

- | | | |
|---|--|---|
| 1 | <code>git tag v1.0.0</code> | # Create a lightweight tag for the current commit |
| 2 | <code>git tag -a v1.0.0 -m "First stable release"</code> | # Create an annotated tag with a message (recommended for releases) |
| 3 | <code>git tag</code> | # List all tags in the repository |
| 4 | <code>git tag -a v1.0.0 f4a7c8e -m "Version 1"</code> | # Create an annotated tag for a specific past commit |
| 5 | <code>git tag -d v1.0.0</code> | # Delete a local tag |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Create a lightweight tag
$ git tag v1.0.0

# Delete a tag for test next command
$ git tag -d v1.0.0
Deleted tag 'v1.0.0' (was e354a8d)

# Create an annotated tag with a message (recommended)
$ git tag -a v1.0.0 -m "First stable release"

# List all tags
$ git tag
v1.0.0

# Delete a tag for test next command
$ git tag -d v1.0.0
Deleted tag 'v1.0.0' (was a5e4d69)

# Tag a past commit using its hash
$ git tag -a v1.0.0 e354a8d -m "version 1"

-----

# to find your commit hash
$ git log --oneline
e354a8d (HEAD -> master) your commit message here
#The hash is the short code on the left side – e354a8d. Use that in your git tag command.
```


Connect to GitHub : git remote

It is used to connect your local project with an online repository like GitHub. It helps you send your code to GitHub (push) and get updates from GitHub (pull), so both local and online versions stay in sync.,

- | | | |
|---|--|-------------------------------------|
| 1 | git remote add origin https://github.com/user/repo.git | # Connect to GitHub one per project |
| 2 | git remote -v | # Verify the connection |
| 3 | git remote set-url origin https://github.com/user/new-repo.git | # Update remote URL if it changes |
| 4 | git remote remove origin | # Remove a remote connection |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Connect to a remote repository (in this case GitHub) do this once per project
$ git remote add origin https://github.com/user/new-repo.git

# Verify the connection
$ git remote -v
origin https://github.com/user/new-repo.git (fetch)
origin https://github.com/user/new-repo.git (push)

# Update remote URL if it changed
$ git remote set-url origin https://github.com/Kumaravi-admin/git-repo.git

$ git remote remove origin # Remove a remote connection
```

Upload Changes : git push

Sends your local commits (changes) to a remote repository like GitHub. It updates the remote branch with your latest work so others can see it or so your online repo stays updated. The basic format is git push <remote> <branch>.

- | | | |
|---|---|---|
| 1 | git push -u origin main | # very first push — set upstream so Git remembers |
| 2 | git push | # Upload changes to a remote repository |
| 3 | git push -u origin feature-login | # Push a new branch for the first time |
| 4 | git push origin --tags | # Push all tags to GitHub |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

$ git remote add origin https://github.com/user/repo.git

# very first push — set upstream so Git remembers
$ git push -u origin main
branch 'main' set up to track 'origin/main'.

# push changes to the remote repository
$ git push
Everything up-to-date

# push a new branch for the first time
$ git push -u origin feature-login
branch 'feature-login' set up to track 'origin/feature-login'.

$ git push origin --tags      # push all tags to the remote repository
Everything up-to-date
```

Download and Merge : **git pull**

Downloads the latest changes from a remote repository and merges them into your current branch. It helps keep your local project up to date with changes made in the remote repository.

- 1 **git pull** # Always do this before starting work
- 2 **git pull origin main** # Pull a specific branch
- 3 **git pull --rebase origin main** # Pull with rebase instead of merge (cleaner history)

```
Git Bash
$ cd /d/LearnCode/Git # Move to the project working directory
-----

# Always pull before starting work
$ git pull
Already up to date.

# Pull a specific branch (in this case main branch)
$ git pull origin main
From https://github.com/Kumaravi-admin/git-repo
* branch      main      -> FETCH_HEAD
Already up to date.

# Download latest changes and keep commit history clean by using rebase
$ git pull --rebase origin main
From https://github.com/Kumaravi-admin/git-repo
* branch      main      -> FETCH_HEAD
Already up to date.
```

Download Without Merging : **git fetch**

Downloads the latest changes, branches, and tags from a remote repository without merging them into your current branch. It lets you review remote updates before deciding whether to merge or rebase them into your work.

- 1 **git fetch origin** # Download all changes from remote does NOT merge
- 2 **git log origin/main --oneline** # See what changed on GitHub before merging
- 3 **git merge origin/main** # Now merge manually when you are ready

```
Git Bash
$ cd /d/LearnCode/Git # Move to the project working directory
-----

# Download the latest changes from the remote repository without merging them
$ git fetch origin

# View the commit history of the remote main branch in one-line format
$ git log origin/main --oneline
ee0fe07 (HEAD -> main, origin/main, origin/feature-login, feature-login) Merge branch 'main' of
https://github.com/Kumaravi-admin/git-repo
d7ea970 Save local changes
e354a8d Initial commit
ad226e4 (feature-payment) Move hello.txt into src folder
cbbd2d4 Remove hello.txt
fe0f063 commit 1
19910e4 Delete hello.txt
b3a3418 Add hello.txt with greeting message
354a51e Add hello.txt with greeting message

# Merge the fetched changes from the remote main branch into the current branch
$ git merge origin/main
Already up to date.
```

Compare Branches : **git diff**

Shows the differences between files, commits, branches, or other Git states.

It helps you compare changes and see what has been added, modified, or removed in a repository.

- | | |
|--|--|
| 1 git diff | # Working directory vs staging area (basic) |
| 2 git diff --staged | # Staging area vs last commit |
| 3 git diff master feature-login | # Compare two branches |
| 4 git diff abc123 def456 | # Compare two specific commits using their hashes |
| 5 git diff --name-only master feature-login | # Show only file names that changed between branches |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# Setup (already covered in previous notes)
git init && echo "Hello Git Learner" > hello.txt && git add . && git commit -m "first commit"

echo "line 2" >> hello.txt

# Working Directory vs Staging Area
$ git diff
@@ -1 +1,2 @@
   Hello Git Learner
+line 2

git add hello.txt

# Staging Area vs Last Commit
$ git diff --staged
@@ -1 +1,2 @@
   Hello Git Learner
+line 2

git commit -m "second commit" && git switch -c feature-login
echo "line 3" >> hello.txt && git add . && git commit -m "feature commit"

# Compare two branches
$ git diff master feature-login
@@ -1,2 +1,3 @@
   Hello Git Learner
   line 2
+line 3

# Show only changed file names
$ git diff --name-only master feature-login
hello.txt

# Compare two commits
$ git log --oneline
6360f6f (HEAD -> feature-login) feature commit
9481fc5 (master) second commit
09d6f82 first commit

$ git diff 6360f6f 9481fc5
@@ -1,3 +1,2 @@
   Hello Git Learner
   line 2
-line 3
```

```
-----
| + Added line | - Removed line | @@ Changed section of the file |
-----
```

Inspect a Commit : **git show**

Displays detailed information about a commit, tag, or other Git object. For commits, it shows the commit details, message, author, date, and the changes (diff) introduced by that commit.

- | | | |
|---|-------------------------------------|---|
| 1 | git show | # See the most recent commit in detail |
| 2 | git show f4a7c8e | # See a specific commit using its hash |
| 3 | git show --name-only f4a7c8e | # See only the files that changed in a commit |
| 4 | git show f4a7c8e:hello.txt | # See a specific file from a past commit |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
```

```
# See the most recent commit in detail
```

```
$ git show
commit 9481fc57cb60f17bc86f10dd276ea8f47f2179ba (HEAD -> master)
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date: Sat May 30 15:15:50 2026 +0530
```

```
second commit
```

```
diff --git a/hello.txt b/hello.txt
index d8096cc..075adae 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 @@
 Hello Git Learner
+line 2
```

```
# Get commit hashes
```

```
$ git log --oneline
9481fc5 (HEAD -> master) second commit
09d6f82 first commit
```

```
# See a specific commit using its hash
```

```
$ git show 09d6f82
commit 09d6f82db43d68dca71a72b23aa26de4b6fbf680
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date: Sat May 30 15:14:56 2026 +0530
```

```
first commit
```

```
diff --git a/hello.txt b/hello.txt
new file mode 100644
index 0000000..d8096cc
--- /dev/null
+++ b/hello.txt
@@ -0,0 +1 @@
+Hello Git Learner
```

```
# See only the files changed in a commit
```

```
$ git show --name-only 09d6f82
commit 09d6f82db43d68dca71a72b23aa26de4b6fbf680
Author: Anjesh Kumar <anjeshravi@yahoo.com>
Date: Sat May 30 15:14:56 2026 +0530
```

```
first commit
```

```
hello.txt
```

```
# See a specific file from a past commit
```

```
$ git show 09d6f82:hello.txt
Hello Git Learner
```

Undo a Pushed Commit : **git revert**

Creates a new commit that reverses the changes made by a previous commit. Unlike git reset, it does not delete commit history, making it the safest way to undo commits that have already been pushed to a remote repository.

- | | |
|--------------------------------------|--|
| 1 git revert HEAD | # Undo the last commit safely |
| 2 git revert f4a7c8e | # Undo a specific past commit using its hash |
| 3 git revert HEAD --no-commit | # Revert without committing immediately |
| 4 git revert --abort | # To cancel the revert before committing |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----
```

```
$ git init
# First commit
$ echo "Line 1" > hello.txt
$ git add .
$ git commit -m "first commit"
```

```
# Second commit
$ echo "Line 2" >> hello.txt
$ git add .
$ git commit -m "second commit"
```

```
# Third commit
$ echo "Line 3" >> hello.txt
$ git add .
$ git commit -m "third commit"
```

```
# View commit history
$ git log --oneline
59b056c (HEAD -> master) third commit
0699abd second commit
672f966 first commit
```

```
# Undo the latest commit by creating a new commit
$ git revert HEAD
[master 2050a02] Revert "third commit"
1 file changed, 1 deletion(-)
```

```
# Undo a specific commit by creating a new commit
$ git revert 0699abd
[master 6446f95] Revert "second commit"
1 file changed, 1 deletion(-)
```

```
# Apply revert changes but don't create a commit yet
$ git revert HEAD --no-commit
```

```
$ git commit -m "Revert login page changes"
[master 239ca2e] Revert login page changes
1 file changed, 1 insertion(+)
```

```
# Fourth commit
$ echo "Line 4" >> hello.txt
$ git add .
$ git commit -m "fourth commit"
```

```
# Start a revert without committing
$ git revert HEAD --no-commit
```

```
# Cancel the revert
$ git revert --abort
```

Clean Up History : git rebase

Moves your commits to a new base commit, creating a cleaner and more linear project history. It is commonly used to update a feature branch with the latest changes from another branch or to combine and reorganize commits before merging.

- | | | |
|---|--|---|
| 1 | git switch feature-login | # Step 1 switch to your feature branch |
| 2 | git rebase main | # Step 2 rebase it on top of main |
| 3 | git add hello.txt
git rebase --continue | # If a conflict happens during rebase: fix conflict, then stage it
continue the rebase |
| 4 | git rebase --abort | # Want to cancel the rebase entirely |
| 5 | git rebase -i HEAD~3 | # Interactive rebase edit last 3 commits |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

$ git init # Initialize a new repository
# Create a file and make the first commit on master
$ echo "Line 1" > hello.txt && git add . && git commit -m "inital commit"
# Create a new branch and switch to it
$ git switch -c feature-login
Switched to a new branch 'feature-login'
# Make two commits on feature-login branch
$ echo "Feautre Line 1" >> hello.txt && git add . && git commit -m "Freature commit 1"
# Second commit on feature branch
$ echo "Feature Line 2" >> hello.txt && git add . && git commit -m "Feature commit 2"
# Switch back to master and make a commit
$ git switch master
$ echo "Main line 1" >> hello.txt && git add . && git commit -m "Main comit 1"

# View commit history as a graph (shows branch divergence)
$ git log --oneline --graph --all
* 6efb589 (HEAD -> master) Main comit 1
| * 52fb54b (feature-login) Feature commit 2
| * 65456e0 Freature commit 1
|/
* 6f4d50a inital commit

# Switch back to feature branch before rebasing
$ git switch feature-login

# Rebase feature branch onto master (replays commits on top of master)
$ git rebase master

# Conflict occurs during first commit replay
Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
error: could not apply 65456e0... Freature commit 1

# After fixing conflict manually, stage resolved file
$ git add hello.txt

# Continue rebase process
$ git rebase --continue
[detached HEAD f857b5a] Freature commit 1
 1 file changed, 4 insertions(+)

# Option to abort rebase if needed
$ git rebase --abort

# Interactive rebase example (edit last commit)
$ git rebase -i HEAD~1
Successfully rebased and updated refs/heads/feature-login.
```

Pick One Commit : **git cherry-pick**

Takes existing commits and applies their changes onto your current branch as new commits.

Your working directory must be clean, and if any conflict happens, Git pauses so you can fix it before continuing.

- | | | |
|---|---|---|
| 1 | git log --oneline feature-login | # First find the commit hash you want |
| 2 | git cherry-pick f4a7c8e | # Apply that one commit to current branch |
| 3 | git cherry-pick f4a7c8e --no-commit | # Apply without committing immediately |
| 4 | git cherry-pick abc123 def456 ghi789 | # Apply multiple commits at once |
| 5 | git cherry-pick -abort | # Cancel a cherry-pick in progress |

```
Git Bash
$ cd /d/LearnCode/Git      # Move to the project working directory
-----

# 1. Initialize repository
git init

# 2. Create base file and commit on master
echo "Base line" > hello.txt
git add .
git commit -m "base commit"

# 3. Create feature branch
git switch -c feature-login

# 4. Add first feature change
echo "Feature A" >> hello.txt
git add .
git commit -m "feature A"

# 5. Add second feature change
echo "Feature B" >> hello.txt
git add .
git commit -m "feature B"

# 6. Go back to master
git switch master

# 7. View feature commits (get hashes)
git log --oneline feature-login
285d3fa (feature-login) feature B
e5c403b feature A
1ff8671 (HEAD -> master) base commit

# 8. Cherry-pick one commit
$ git cherry-pick 285d3fa
[master 0032118] feature B
Date: Sat May 30 19:10:53 2026 +0530
1 file changed, 1 insertion(+)

# 9. Cherry-pick multiple commits
$ git cherry-pick e5c403b 285d3fa

# 10. Cherry-pick without committing immediately
$ git cherry-pick --no-commit e5c403b

# 11. If conflict happens: fix file then continue
git add hello.txt
git cherry-pick --continue

# 12. If you want to cancel cherry-pick
git cherry-pick -abort
```

Ignore Files : .gitignore

A .gitignore file specifies intentionally untracked files that Git should ignore. Files already tracked by Git are not affected. Each line in a .gitignore file specifies a pattern — Git checks each untracked file against these patterns before deciding whether to track it.

1 touch .gitignore

Create the file in your project root

```
# dependencies - never commit these
node_modules/

# environment variables - passwords live here
.env
.env.local

# log files
*.log

# operating system files
.DS_Store
Thumbs.db

# build output folders
dist/
build/
```

Commit the .gitignore file

```
admin@DESKTOP-MMM7LA2 MINGW64 /d/LearnCode/Git (master)
$ touch .gitignore

admin@DESKTOP-MMM7LA2 MINGW64 /d/LearnCode/Git (master)
$

admin@DESKTOP-MMM7LA2 MINGW64 /d/LearnCode/Git (master)
$ git add .gitignore

admin@DESKTOP-MMM7LA2 MINGW64 /d/LearnCode/Git (master)
$ git commit -m "Add .gitignore"
[master d0b79ce] Add .gitignore
1 file changed, 17 insertions(+)
create mode 100644 .gitignore
```


Workflow : Local

This workflow explains a basic Git process for solo development. You start by initializing a repository, track and save changes using commits, and use branches to build new features safely. After completing the feature, you merge it back into the main branch and remove the feature branch to keep the project clean and organized.

```
1. Workflow Local (Solo, No GitHub)
-----
# step 1 - initialize repository
git init

# step 2 - create your files and work...

# step 3 - check what changed
git status
git diff

# step 4 - stage your changes
git add .

# step 5 - save your work
git commit -m "What you did"

# step 6 - create a branch for new feature
git switch -c feature-dark-mode

# step 7 - work on feature...
git add .
git commit -m "Add dark mode toggle"

# step 8 - merge back into main
git switch main
git merge feature-dark-mode

# step 9 - clean up branch
git branch -d feature-dark-mode
-----
```

Workflow : Remote

This workflow shows how to use Git with GitHub for a solo project. You connect your repo to GitHub, push your code, and keep syncing changes as you work. Features are developed in branches, merged into main, and pushed again to GitHub. Unused branches are deleted to keep things clean.

```
2. Workflow Remote (Solo, With GitHub)
-----
# step 1 - connect local repo to GitHub (do this once)
git remote add origin https://github.com/user/repo.git

# step 2 - first push to GitHub
git push -u origin main

# step 3 - do your work and commit...
git add .
git commit -m "What you did"

# step 4 - push to GitHub
git push

# step 5 - create a branch, work, merge
git switch -c feature-dark-mode
git add .
git commit -m "Add dark mode toggle"
git switch main
git merge feature-dark-mode

# step 6 - push updated main to GitHub
git push

# step 7 - clean up
git branch -d feature-dark-mode
-----
```

Workflow : Team

This workflow shows how teams work with Git and GitHub. You always start by pulling the latest code, then create a feature branch for your task. After working and committing changes, you update your branch with the latest main branch using rebase to avoid conflicts. Once everything is ready, you merge your feature into main, push it to GitHub, and delete the branch to keep the repository clean.

3. Workflow - Team (With GitHub)

```
-----
# step 1 – always pull latest changes first
git pull

# step 2 – create your own branch for the task
git switch -c feature-dark-mode

# step 3 – do your work...
git add .
git commit -m "Add dark mode toggle"

# step 4 – before merging get latest main changes
git switch main
git pull
git switch feature-dark-mode

# step 5 – rebase your branch on top of latest main
git rebase main

# step 6 – fix conflicts if any
git add hello.txt
git rebase --continue

# step 7 – merge into main
git switch main
git merge feature-dark-mode

# step 8 – push to GitHub
git push

# step 9 – clean up branch
git branch -d feature-dark-mode
-----
```